



Universidad Técnica de Ambato

Facultad de Ingeniería en Sistemas, Electrónica e Industrial

Carrera de Software



Sistema Básico de Control de Estudiantes y Calificaciones:

Aplicación de Clases, Objetos y Métodos en C++ y Java

Asignatura: Algoritmos y Lógica de Programación

Docente: José Ruben Caiza

Nombre: Chiluiza Quishpe Diego Steed

APE04 – Guía Práctica

Mayo 2025



Resumen

El presente informe corresponde a la práctica APE04 de la asignatura Algoritmos y Lógica de Programación de la carrera de Software de la Universidad Técnica de Ambato. El objetivo de la práctica fue desarrollar un sistema básico de control de estudiantes y calificaciones implementado en dos lenguajes de programación: C++ y Java. Para ello se aplicaron los principios fundamentales de la programación orientada a objetos, específicamente el diseño de clases con atributos privados, constructores, métodos de acceso (getters y setters) y métodos de lógica para calcular promedios y determinar el estado académico de cada estudiante. El sistema permite registrar un mínimo de cinco estudiantes, ingresar tres notas por cada uno, calcular automáticamente el promedio, validar que las notas estén en el rango de 0 a 10 y mostrar un resumen con la cantidad de estudiantes aprobados y reprobados. La condición de aprobación establecida es un promedio mayor o igual a 7.00.

Palabras clave: programación orientada a objetos, clases, objetos, métodos, encapsulación, C++, Java.

Introducción

La programación orientada a objetos (POO) es uno de los paradigmas más utilizados en el desarrollo de software moderno. Su popularidad se debe a que permite organizar el código de manera modular, reducir la repetición y facilitar el mantenimiento y la escalabilidad de las aplicaciones. De acuerdo con Deitel y Deitel (2017), la POO modela el software como un conjunto de objetos que interactúan entre sí, cada uno con su propio estado y comportamiento.

En este contexto, la práctica APE04 tiene como propósito aplicar los conceptos de clases, objetos y métodos para construir un sistema de control de calificaciones estudiantiles. El sistema fue implementado en C++ y Java, dos de los lenguajes más representativos del paradigma orientado a objetos, lo cual permite comparar su sintaxis y verificar que los principios fundamentales se mantienen independientemente del lenguaje utilizado.

La relevancia de esta práctica radica en que el diseño correcto de una clase, con encapsulación adecuada y métodos bien definidos, constituye la base sobre la que se construyen sistemas de software de mayor complejidad. Stroustrup (2013) señala que la abstracción de datos mediante clases es la característica que diferencia a C++ de su predecesor C, y Bloch (2018) destaca que en Java la escritura de clases bien encapsuladas es la piedra angular de un diseño robusto.

Objetivos

Objetivo General

Desarrollar un programa en C++ y Java que gestione el registro de estudiantes y sus calificaciones, aplicando los principios de la programación orientada a objetos mediante el uso de clases, objetos y métodos.

Objetivos Específicos

- Definir e implementar la clase Estudiante con atributos privados, constructor, getters, setters y métodos de lógica en C++ y Java.
- Registrar un mínimo de cinco estudiantes con sus respectivas tres notas.
- Calcular automáticamente el promedio de cada estudiante mediante el método `calcularPromedio()`.
- Determinar el estado académico (Aprobado/Reprobado) con la condición $\text{promedio} \geq 7.00$.
- Validar que cada nota ingresada se encuentre en el rango $[0, 10]$.
- Mostrar un listado completo de estudiantes con el resumen de aprobados y reprobados.
- Subir el proyecto a un repositorio GitHub con al menos tres commits organizados.

Marco Teórico

Programación Orientada a Objetos

La programación orientada a objetos es un paradigma de diseño de software que organiza el código en torno a objetos, los cuales combinan estado (datos) y comportamiento (funciones). Según Deitel y Deitel (2016), los cuatro pilares de la POO son la encapsulación, la herencia, el polimorfismo y la abstracción. Esta práctica se enfoca principalmente en los conceptos de encapsulación y abstracción, aplicados mediante el diseño de la clase Estudiante.

Clases y Objetos

Una clase es una plantilla o molde que describe la estructura y el comportamiento de un conjunto de objetos. Un objeto es una instancia concreta de una clase. Stroustrup (2013) define una clase en C++ como un tipo definido por el usuario que puede contener datos miembro y funciones miembro, los cuales determinan el estado y las operaciones del objeto. En Java, el concepto es equivalente: toda entidad del sistema se modela como un objeto perteneciente a una clase (Deitel y Deitel, 2017).

En este proyecto, la clase Estudiante define los atributos cédula, nombre, apellido, nota1, nota2, nota3, promedio y estado. Cada estudiante registrado en el programa es un objeto de esta clase, con sus propios valores para dichos atributos.

Encapsulación

La encapsulación consiste en restringir el acceso directo a los atributos de una clase declarándolos como privados, y exponer únicamente la interfaz necesaria a través de métodos públicos. Bloch (2018) afirma que la encapsulación es probablemente el principio más importante del diseño orientado a objetos, porque permite modificar la implementación interna de una clase sin afectar el código que la utiliza. En este proyecto, todos los atributos de la clase Estudiante son privados y se accede a ellos únicamente mediante getters y setters.

Métodos

Los métodos son las operaciones que puede realizar un objeto. En la clase Estudiante se implementaron los siguientes métodos con responsabilidades claramente diferenciadas: `calcularPromedio()`, que obtiene el promedio aritmético de las tres notas; `determinarEstado()`, que asigna el estado académico según el promedio; y `mostrarInfo()`, que imprime los datos del estudiante en la consola. Esta separación de responsabilidades es coherente con el principio de responsabilidad única descrito por Bloch (2018).

Validación de Entradas

La validación de entradas es una práctica fundamental para garantizar la integridad de los datos en cualquier sistema de software. En este proyecto se implementó mediante un bucle `do-while` que impide avanzar en el registro si la nota ingresada está fuera del rango permitido [0,

10]. Deitel y Deitel (2016) recomiendan que toda entrada de usuario sea validada antes de ser procesada, especialmente cuando su valor se usará en cálculos posteriores.

Desarrollo del Programa

Estructura de la Clase Estudiante

La siguiente tabla presenta los componentes diseñados para la clase Estudiante en ambos lenguajes de programación.

Link de GIT-HUB: https://github.com/diegosteed/APE04_Individual

Tabla 1

Estructura de la clase Estudiante

Componente	Identificador	Descripción
Atributo	cedula, nombre, apellido	Datos de identificación personal
Atributo	nota1, nota2, nota3	Calificaciones de cada parcial (0–10)
Atributo	promedio	Media aritmética calculada automáticamente
Atributo	estado	Aprobado si promedio ≥ 7.00 , de lo contrario Reprobado
Constructor	Estudiante(...)	Inicializa todos los atributos y calcula promedio y estado
Método	calcularPromedio()	Calcula $(nota1 + nota2 + nota3) / 3$
Método	determinarEstado()	Asigna estado según el valor del promedio
Método	mostrarInfo()	Imprime todos los datos del estudiante en consola
Getters / Setters	getCedula(), setNota1()...	Acceso controlado a los atributos privados

Nota. Elaboración propia.

Implementación en C++

A continuación se presenta el código fuente del programa desarrollado en C++. La clase Estudiante fue definida en el mismo archivo main.cpp para simplificar la compilación, conforme a las indicaciones de la guía de práctica.

```
Ape04_Ejercicio.cpp U X
APE04 > C: Ape04_Ejercicio.cpp > ...
1  /*
2   * APE04 - Clases, Objetos y Métodos
3   * Asignatura: Algoritmos y Logica de Programacion
4   * Carrera: Software - Universidad Tecnica de Ambato
5   * Docente: Jose Ruben Caiza
6   */
7  #include <iostream>
8  #include <string>
9  #include <iomanip>
10 using namespace std;
11
12 class Estudiante {
13 private:
14     string cedula, nombre, apellido;
15     double nota1, nota2, nota3;
16     double promedio;
17     string estado;
18
19 public:
20     // Constructor
21     Estudiante(string ced, string nom, string ape,
22               double n1, double n2, double n3) {
23         cedula=ced; nombre=nom; apellido=ape;
24         nota1=n1; nota2=n2; nota3=n3;
25         calcularPromedio();
26         determinarEstado();
27     }
28     // Getters
29     string getCedula() { return cedula; }
30     string getNombre() { return nombre; }
31     double getPromedio() { return promedio; }
32     string getEstado() { return estado; }
33     // Setters
34     void setNota1(double n) { nota1=n; calcularPromedio(); determinarEstado(); }
35     void setNota2(double n) { nota2=n; calcularPromedio(); determinarEstado(); }
36     void setNota3(double n) { nota3=n; calcularPromedio(); determinarEstado(); }
37     // Calcula el promedio de las tres notas
38     void calcularPromedio() {
```



```
39     promedio = (nota1 + nota2 + nota3) / 3.0;
40 }
41 // Determina estado segun promedio >= 7.00
42 void determinarEstado() {
43     estado = (promedio >= 7.00) ? "Aprobado" : "Reprobado";
44 }
45 // Muestra la informacion completa del estudiante
46 void mostrarInfo() {
47     cout << "Cedula : " << cedula << endl;
48     cout << "Nombre : " << nombre << " " << apellido << endl;
49     cout << fixed << setprecision(2);
50     cout << "Promedio: " << promedio << endl;
51     cout << "Estado : " << estado << endl;
52     cout << endl;
53 }
54 };
55
56 // Funcion para leer nota validada en rango [0, 10]
57 double leerNota(const string& msg) {
58     double n;
59     do {
60         cout << msg; cin >> n;
61         if (n < 0 || n > 10) cout << "Nota invalida. Rango permitido: 0 - 10\n";
62     } while (n < 0 || n > 10);
63     return n;
64 }
65
66 int main() {
67     const int TOTAL = 5;
68     Estudiante* lista[TOTAL];
69
70     for (int i = 0; i < TOTAL; i++) {
71         string ced, nom, ape;
72         cout << "Cedula : "; cin >> ced;
73         cout << "Nombre : "; cin >> nom;
74         cout << "Apellido: "; cin >> ape;
75         double n1 = leerNota("Nota 1: ");
76         double n2 = leerNota("Nota 2: ");
77         double n3 = leerNota("Nota 3: ");
78         lista[i] = new Estudiante(ced, nom, ape, n1, n2, n3);
79     }
80
81     int aprobados = 0, reprobados = 0;
82     for (int i = 0; i < TOTAL; i++) {
83         lista[i]->mostrarInfo();
84         if (lista[i]->getEstado() == "Aprobado") aprobados++;
85         else reprobados++;
86         delete lista[i];
87     }
88     cout << "Aprobados : " << aprobados << endl;
89     cout << "Reprobados: " << reprobados << endl;
90     return 0;
91 }
92
```

Pruebas de Casos. –



```
Nombre : Pedro
Apellido: Chiluiza
Nota 1: 2
Nota 2: 5
Nota 3: 7
Cedula : 12345678
Nombre : Steed Chiluiza
Promedio: 7.00
Estado : Aprobado

Cedula : 23452878
Nombre : Lucas Caiza
Promedio: 7.17
Estado : Aprobado

Cedula : 87620234
Nombre : Fernando Flore
Promedio: 7.33
Estado : Aprobado

Cedula : Robert
Nombre : Armas Armas
Promedio: 5.77
Estado : Reprobado

Cedula : 8
Nombre : Pedro Chiluiza
Promedio: 4.67
Estado : Reprobado

Aprobados : 3
Reprobados: 2
PS C:\Users\Steed\OneDrive\Documentos\Trabajos _ Licencia _ C\Proyecto_Final_Programacion\ExamenUnidad3\APE04\output>S
```

Pseudocódigo. –

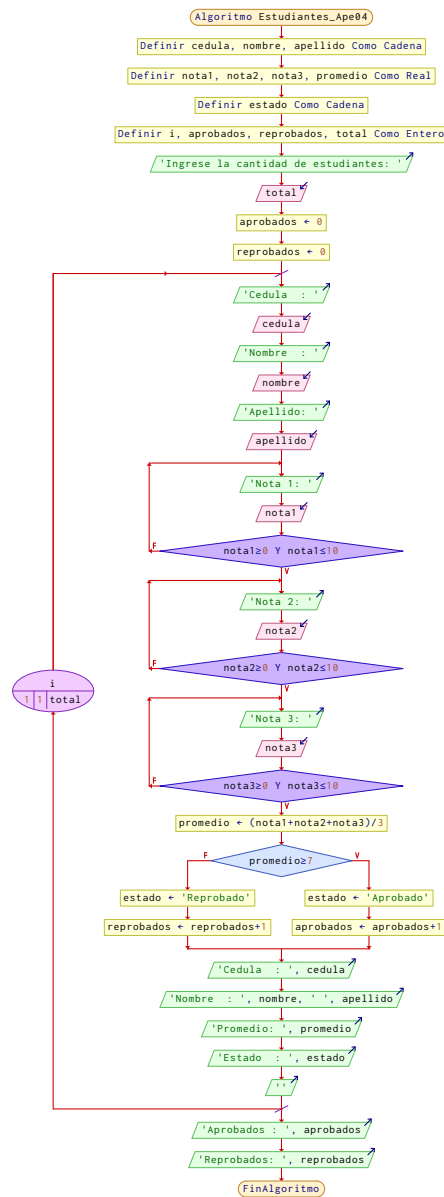
Algoritmo Estudiantes_Ape04

```
Definir cedula, nombre, apellido Como Cadena
Definir nota1, nota2, nota3, promedio Como Real
Definir estado Como Cadena
Definir i, aprobados, reprobados, total Como Entero
Escribir 'Ingrese la cantidad de estudiantes: 'Sin Saltar
Leer total
aprobados <- 0
reprobados <- 0
Para i<-1 Hasta total Con Paso 1 Hacer
    Escribir 'Cedula : 'Sin Saltar
    Leer cedula
    Escribir 'Nombre : 'Sin Saltar
    Leer nombre
    Escribir 'Apellido: 'Sin Saltar
    Leer apellido
    Repetir
        Escribir 'Nota 1: 'Sin Saltar
        Leer nota1
    Hasta Que nota1>=0 Y nota1<=10
    Repetir
        Escribir 'Nota 2: 'Sin Saltar
        Leer nota2
```



```
Hasta Que nota2>=0 Y nota2<=10
Repetir
    Escribir 'Nota 3: 'Sin Saltar
    Leer nota3
Hasta Que nota3>=0 Y nota3<=10
promedio <- (nota1+nota2+nota3)/3
Si promedio>=7 Entonces
    estado <- 'Aprobado'
    aprobados <- aprobados+1
SiNo
    estado <- 'Reprobado'
    reprobados <- reprobados+1
FinSi
Escribir 'Cedula : ', cedula
Escribir 'Nombre : ', nombre, ' ', apellido
Escribir 'Promedio: ', promedio
Escribir 'Estado : ', estado
Escribir "
FinPara
Escribir 'Aprobados : ', aprobados
Escribir 'Reprobados: ', reprobados
FinAlgoritmo
```

Diagrama de Flujo. –



Implementación en Java — Clase Estudiante

En Java el proyecto fue dividido en dos archivos: Estudiante.java, que contiene la definición de la clase, y Main.java, que contiene el método principal del programa.

```
J Ape04_Ejercicio.java U • Notas de la versión: 1.121.0
J Ape04_Ejercicio.java
1  /**
2   * APE04 - Clases, Objetos y Metodos
3   * Clase Estudiante: encapsula datos y comportamientos
4   */
5   public class Estudiante {
6       // Atributos privados
7       private String cedula, nombre, apellido;
8       private double nota1, nota2, nota3;
9       private double promedio;
10      private String estado;
11
12      // Constructor
13      public Estudiante(String cedula, String nombre, String apellido,
14          double nota1, double nota2, double nota3) {
15          this.cedula = cedula;
16          this.nombre = nombre;
17          this.apellido = apellido;
18          this.nota1 = nota1;
19          this.nota2 = nota2;
20          this.nota3 = nota3;
21          calcularPromedio();
22          determinarEstado();
23      }
24      // Getters
25      public String getCedula() { return cedula; }
26      public String getNombre() { return nombre; }
27      public double getPromedio() { return promedio; }
28      public String getEstado() { return estado; }
29      // Setters con recalcu automatic
30      public void setNota1(double n) { nota1=n; calcularPromedio(); determinarEstado(); }
31      public void setNota2(double n) { nota2=n; calcularPromedio(); determinarEstado(); }
32      public void setNota3(double n) { nota3=n; calcularPromedio(); determinarEstado(); }
33      // Calcula el promedio de las tres notas
34      public void calcularPromedio() {
35          this.promedio = (nota1 + nota2 + nota3) / 3.0;
36      }
37      // Determina estado segun promedio >= 7.00
38      public void determinarEstado() {
39          this.estado = (promedio >= 7.00) ? "Aprobado" : "Reprobado";
40      }
41      // Imprime la informacion del estudiante
42      public void mostrarInfo() {
43          System.out.printf("Cedula : %s\n", cedula);
44          System.out.printf("Nombre : %s %s\n", nombre, apellido);
45          System.out.printf("Promedio: %.2f\n", promedio);
46          System.out.printf("Estado : %s\n", estado);
47      }
48  }
```

Implementación en Java — Clase Main



```
J EjercicioEstudiante.java U • J EjercicioMenu.java U •
J EjercicioMenu.java
1  import java.util.Scanner;
2  /**
3   * Clase Main: punto de entrada del sistema de calificaciones
4   */
5  public class Main {
6      // Lee y valida una nota en rango [0, 10]
7      public static double leerNota(Scanner sc, String msg) {
8          double nota;
9          do {
10             System.out.print(msg);
11             nota = sc.nextDouble();
12             if (nota < 0 || nota > 10)
13                 System.out.println("Nota invalida. Rango permitido: 0 - 10");
14             while (nota < 0 || nota > 10);
15             return nota;
16         }
17     }
18     public static void main(String[] args) {
19         Scanner sc = new Scanner(System.in);
20         final int TOTAL = 5;
21         Estudiante[] lista = new Estudiante[TOTAL];
22         for (int i = 0; i < TOTAL; i++) {
23             System.out.print("Cedula : "); String ced = sc.next();
24             System.out.print("Nombre : "); String nom = sc.next();
25             System.out.print("Apellido: "); String ape = sc.next();
26             double n1 = leerNota(sc, "Nota 1: ");
27             double n2 = leerNota(sc, "Nota 2: ");
28             double n3 = leerNota(sc, "Nota 3: ");
29             lista[i] = new Estudiante(ced, nom, ape, n1, n2, n3);
30         }
31         int aprobados = 0, reprobados = 0;
32         for (Estudiante e : lista) {
33             e.mostrarInfo();
34             if (e.getEstado().equals("Aprobado")) aprobados++;
35             else reprobados++;
36         }
37         System.out.println("Aprobados : " + aprobados);
38         System.out.println("Reprobados: " + reprobados);
39         sc.close();
40     }
41 }
```

Marco Teórico

El desarrollo de software moderno exige estructuras de código organizadas, mantenibles y reutilizables. Para lograrlo, el paradigma de la programación orientada a objetos se ha consolidado como uno de los enfoques más adoptados en la industria, ya que permite modelar entidades del mundo real como estructuras de datos con comportamiento propio. Según Deitel y Deitel (2017), este paradigma organiza el software en torno a objetos que combinan estado y comportamiento, lo que reduce la complejidad de los sistemas y facilita su escalabilidad.

El concepto fundamental sobre el que descansa este proyecto es el de clase. Una clase es una plantilla que define los atributos y métodos que tendrán los objetos creados a partir de ella. Stroustrup (2013) la describe como un tipo definido por el usuario que encapsula datos y las operaciones que actúan sobre ellos. En este proyecto, la clase Estudiante representa esa plantilla, y cada estudiante registrado en el sistema es un objeto, es decir, una instancia concreta de dicha clase con sus propios valores de cédula, nombre y calificaciones.

Uno de los principios que guía el diseño de la clase es la encapsulación, que consiste en declarar los atributos como privados y controlar su acceso mediante métodos públicos conocidos como getters y setters. Bloch (2018) sostiene que la encapsulación es el principio más importante del diseño orientado a objetos, porque permite modificar la implementación interna de una clase sin afectar el código externo que la utiliza. Gracias a este principio, los datos del estudiante quedan protegidos de modificaciones accidentales y la lógica de cálculo del promedio y del estado académico permanece centralizada dentro de la propia clase.

Finalmente, los métodos son las operaciones que definen el comportamiento de los objetos. En la clase Estudiante se implementaron métodos con responsabilidades claramente diferenciadas: `calcularPromedio()` obtiene la media aritmética de las tres notas, `determinarEstado()` evalúa si el estudiante aprueba o reprueba con base en la condición de promedio mayor o igual a 7.00, y `mostrarInfo()` presenta los datos completos en consola.

Esta separación de responsabilidades, conocida en ingeniería de software como cohesión, garantiza que cada parte del código tenga un propósito único y sea fácil de probar, modificar y extender (Deitel y Deitel, 2016).

Análisis de Resultados

Funcionamiento del Sistema

El sistema solicita al usuario los datos de identificación y las tres calificaciones de cada estudiante. Una vez ingresadas las notas, el constructor invoca automáticamente los métodos `calcularPromedio()` y `determinarEstado()`, lo que garantiza que el objeto siempre se encuentre en un estado consistente. Este comportamiento evita que el sistema opere con promedios desactualizados, un error frecuente cuando el cálculo se delega al programa principal en lugar de encapsularse dentro de la clase.

La validación de notas mediante el bucle `do-while` impide que el programa procese valores fuera del rango `[0, 10]`. Este control es especialmente importante porque las notas son la

base del cálculo del promedio y de la determinación del estado del estudiante; un dato inválido propagaría un error a todos los atributos derivados.

Comparación entre C++ y Java

Ambas implementaciones comparten la misma estructura lógica: clase con atributos privados, constructor, getters, setters y métodos de negocio. La principal diferencia sintáctica radica en que en C++ los objetos pueden crearse en el stack o en el heap, mientras que en Java todos los objetos se crean en el heap mediante el operador new. Por ello, en C++ fue necesario liberar la memoria con delete al finalizar el programa, responsabilidad que en Java es gestionada automáticamente por el recolector de basura (Deitel y Deitel, 2016; Deitel y Deitel, 2017).

Ventajas de la Programación Orientada a Objetos en Este Proyecto

- Encapsulación: los atributos privados protegen los datos de modificaciones accidentales desde fuera de la clase.
- Cohesión: cada método tiene una responsabilidad única y claramente definida, lo que facilita las pruebas y el mantenimiento.
- Reutilización: la misma clase Estudiante es instanciada tantas veces como sea necesario sin duplicar código.
- Consistencia: el constructor y los setters garantizan que el promedio y el estado siempre estén actualizados.

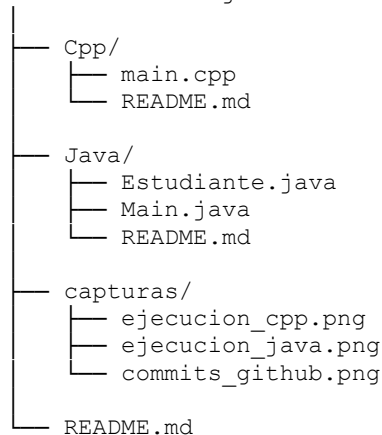
- Escalabilidad: agregar nuevos atributos o métodos a la clase no requiere modificar el programa principal.

Repositorio GitHub

Estructura de Carpetas

El repositorio fue organizado con la siguiente estructura de directorios, conforme a los lineamientos de la guía APE04:

APE04-Clases-Objetos-Metodos/



Commits Realizados

Se realizaron cuatro commits organizados con mensajes descriptivos que reflejan el avance progresivo del proyecto:

Commit	Mensaje	Descripción
#1	feat: add Estudiante class C++	Clase con atributos, constructor y métodos en C++
#2	feat: add Estudiante class Java	Implementación equivalente en Java
#3	feat: add Main and validations	Programa principal con validación de notas
#4	docs: add README and captures	Documentación y capturas de ejecución

Nota. Elaboración propia.

Conclusiones

La práctica APE04 permitió verificar que la programación orientada a objetos ofrece ventajas concretas sobre el paradigma procedural al momento de diseñar sistemas que manejan entidades con estado y comportamiento propios. La clase Estudiante concentró toda la lógica del negocio, lo que hizo que el programa principal resultara más limpio, legible y fácil de extender.

La implementación en C++ y Java confirmó que los conceptos de clase, encapsulación y métodos son independientes del lenguaje de programación. Si bien la sintaxis varía, la estructura lógica del diseño orientado a objetos es la misma en ambos casos, lo que evidencia la universalidad del paradigma.

La validación de entradas mediante bucles do-while demostró ser una estrategia efectiva para proteger la integridad de los datos antes de procesarlos. Este hábito de programación defensiva es especialmente relevante en sistemas reales donde las entradas del usuario no siempre son confiables.

El uso de GitHub para el control de versiones, con commits organizados y mensajes descriptivos, refuerza hábitos profesionales de desarrollo de software que son esenciales en entornos de trabajo colaborativo.



Recomendaciones

- Comentar el código de manera clara y concisa, explicando la responsabilidad de cada método y el propósito de los atributos principales, tal como lo establecen las buenas prácticas de programación descritas por Bloch (2018).
- Utilizar nombres de variables y métodos en un idioma consistente a lo largo del proyecto para mejorar su legibilidad y facilitar el trabajo colaborativo.
- Realizar commits frecuentes con mensajes que sigan convenciones establecidas como Conventional Commits, lo que facilita la trazabilidad del historial del proyecto.
- Ampliar el sistema incorporando búsqueda de estudiantes por cédula, edición de notas y persistencia de datos en archivos de texto, como extensiones naturales de la práctica.
- Verificar que el repositorio GitHub sea público antes de copiar el enlace para entregarlo en Moodle.



Referencias

Bloch, J. (2018). *Effective Java (3.^a ed.)*. Addison-Wesley

Deitel, P. J., & Deitel, H. M. (2016). *C++ How to Program (10.^a ed.)*. Pearson Education

Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program, Early Objects (11.^a ed.)*. Pearson Education

Stroustrup, B. (2013). *The C++ Programming Language (4.^a ed.)*. Addison-Wesley

Universidad Técnica de Ambato. (2025). *Guía de práctica APE04: Clases, objetos y métodos*.
Facultad de Ingeniería en Sistemas, Electrónica e Industrial